

DWDM Lab Master Setup & Prompting Guide

Complete Environment Requirements, Tech Stack, & AI Agent Prompts for Experiments 1 through 7

1. Tech Stack & Environment Setup (Experiments 1 to 7)

Exp	Experiment Name	Core Technology / Stack	Primary Libraries / Modules
Exp 1	Data Warehouse Schema Design	SQL / Database Engine	<code>sqlite3</code> (Python built-in) or pure SQL DDL
Exp 2	OLAP Operations using SQL	SQL RDBMS (MySQL / Postgres / SQLite)	<code>sqlite3</code> (Python built-in) or pure SQL queries
Exp 3	Decision Tree Classification	Python 3.8+	<code>pandas</code> , <code>numpy</code> , <code>scikit-learn</code> , <code>matplotlib</code>
Exp 4	Simple Linear Regression	Python 3.8+	<code>pandas</code> , <code>numpy</code> , <code>scikit-learn</code> , <code>matplotlib</code> , <code>seaborn</code>
Exp 5	K-Means Clustering	Python 3.8+	<code>pandas</code> , <code>numpy</code> , <code>scikit-learn</code> , <code>matplotlib</code> , <code>seaborn</code>
Exp 6	Hierarchical Clustering	Python 3.8+	<code>pandas</code> , <code>numpy</code> , <code>scipy</code> , <code>scikit-learn</code> , <code>matplotlib</code>
Exp 7	Apriori Algorithm (Market Basket)	Python 3.8+	<code>pandas</code> , <code>mlxtend</code>

Installation Commands

Local Machine (VS Code / Terminal / Jupyter Notebook)

Run this command in your terminal to install all Python dependencies across all experiments:

```
pip install pandas numpy scikit-learn scipy matplotlib seaborn mlxtend
```

Note: `sqlite3` is built directly into Python for Experiments 1 and 2.

Google Colab (Zero Setup Cloud Notebook)

All required libraries except `mlxtend` come pre-installed in Google Colab. Simply run this at the top of your notebook for Experiment 7:

```
!pip install mlxtend
```

2. Detailed AI Prompts (Exp 1 to Exp 7)

Copy and paste any prompt below into an AI agent (e.g., ChatGPT, Claude, DeepSeek) to generate the complete code, step-by-step code explanations, numerical outputs, and viva questions without providing uploaded source files.

Experiment 1: Data Warehouse Schema Design Prompt

Act as a Data Warehousing lab instructor. Provide a complete, production-ready lab writeup and SQL code for Experiment 1: Data Warehouse Schema Design (Star Schema vs Snowflake Schema).

Structure your response with the following exact sections:

1. AIM: A clear 2-sentence objective statement.
2. THEORY & CONCEPTS:
 - Detailed explanation of Fact Tables vs Dimension Tables.
 - Conceptual comparison between Star Schema, Snowflake Schema, and Fact Constellation Schema.
3. COMPLETE EXECUTABLE SQL SCRIPT:
 - Provide clean, standard SQL DDL queries to create a fully functional Star Schema (1 central Fact table with 3 surrounding Dimension tables: Student/Customer, Time/Date, Product/Subject).
 - Provide SQL DDL queries to convert one dimension into a normalized Snowflake Schema (e.g., separating Location from Student into a sub-dimension).
 - Include sample INSERT INTO statements with realistic dummy data.
4. LINE-BY-LINE CODE EXPLANATION:
 - Explain the primary keys, foreign key constraints, metrics (measures), and normalization choices step-by-step.
5. STEP-BY-STEP EXPECTED OUTPUTS:
 - Show the exact tabular structure (schema diagram equivalent represented in formatted Markdown tables) resulting from your SQL execution.
6. TOP 5 LAB VIVA QUESTIONS & SHORT ANSWERS:
 - Provide the 5 most common viva questions on schema design with concise, 1-2 sentence answers.

Experiment 2: OLAP Operations (SQL) Prompt

Act as a Data Warehousing lab instructor. Provide a complete academic lab writeup and SQL code for Experiment 2: Execution of OLAP Operations (Roll-up, Drill-down, Slice, Dice, and Pivot) using SQL.

Structure your response with the following exact sections:

1. AIM: A 2-sentence objective statement.
2. THEORY & CONCEPTS:
 - Detailed mathematical/logical definition of OLAP operations: Roll-up, Drill-down, Slice, Dice, and Pivot.
3. COMPLETE EXECUTABLE SQL SCRIPT:
 - Write standard SQL DDL to set up a sample Data Warehouse Fact table (`Sales\_Fact` or `Performance\_Fact`) with dimensions: Time (Year, Quarter, Month), Location (Country, City), and Product/Branch.
 - Insert representative sample data (at least 10 rows).
 - Provide individual, clearly labeled SQL queries demonstrating:
 - a) Roll-up (using GROUP BY with ROLLUP or multi-level aggregation)
 - b) Drill-down (moving from high-level summary to granular details)
 - c) Slice (filtering on 1 dimension, e.g., Quarter = 'Q1')
 - d) Dice (filtering on 2+ dimensions simultaneously)
 - e) Pivot (rotating rows to columns using conditional aggregation CASE WHEN / SUM).
4. CODE & QUERY EXPLANATION:
 - Explain how each SQL clause maps directly to its corresponding OLAP operation step-by-step.
5. STEP-BY-STEP EXPECTED OUTPUTS:
 - Show the exact resulting tables (in clean Markdown table format) that each of the 5 OLAP queries will output.
6. TOP 5 LAB VIVA QUESTIONS & SHORT ANSWERS:
 - Include 5 viva questions on OLAP cubes, MOLAP vs ROLAP, and query performance with concise answers.

Experiment 3: Classification using Decision Trees Prompt

Act as a Machine Learning lab instructor. Provide a complete academic lab report and fully functional Python code for Experiment 3: Classification using Decision Trees.

Structure your response with the following exact sections:

1. AIM: A concise 2-sentence objective statement.
2. THEORY & MATHEMATICAL FORMULAS:
 - Explain Entropy and Information Gain with their explicit mathematical formulas:
Entropy: $H(S) = - \sum(p_i * \log_2(p_i))$
Information Gain: $\text{Gain}(S, A) = H(S) - \sum(|S_v|/|S| * H(S_v))$
 - Briefly contrast Information Gain (ID3) with Gini Impurity (CART).
3. COMPLETE STANDALONE PYTHON CODE:
 - Use ``pandas`, `numpy`, `scikit-learn` (`DecisionTreeClassifier`, `train_test_split`, `accuracy_score`, `confusion_matrix`, `plot_tree`), and `matplotlib`.`
 - Embed a small, clean synthetic dataset (e.g., "PlayTennis" or "Student Exam Pass/Fail") directly inside the code using a pandas DataFrame so it runs standalone without external CSV files.
 - Preprocess categorical features using LabelEncoding or One-Hot Encoding.
 - Train the ``DecisionTreeClassifier(criterion='entropy')``.
 - Output tree visualization using ``matplotlib``.
4. STEP-BY-STEP CODE EXPLANATION:
 - Explain how data loading, feature encoding, model training, tree split evaluation, and metric calculation happen line-by-line.
5. EXPECTED OUTPUTS & INTERPRETATION:
 - Provide the exact numbers produced by the execution: sample evaluation metrics, Confusion Matrix array, and textual representation of the decision rules at each tree node.
6. TOP 5 LAB VIVA QUESTIONS & SHORT ANSWERS:
 - Include 5 frequently asked viva questions (e.g., overfitting, pruning, Information Gain selection) with concise 1-2 sentence answers.

Experiment 4: Simple Linear Regression Prompt

Act as a Machine Learning lab instructor. Provide a complete academic lab report and fully functional Python script for Experiment 4: Simple Linear Regression.

Structure your response with the following exact sections:

1. AIM: A concise 2-sentence objective statement.
2. THEORY & MATHEMATICAL FORMULAS:
 - Explain Simple Linear Regression modeling $Y = \alpha + \beta * X$.
 - Provide closed-form equations for slope (β), intercept (α), Mean Squared Error (MSE), and Coefficient of Determination (R^2 score).
3. COMPLETE STANDALONE PYTHON CODE:
 - Use ``pandas`, `numpy`, `scikit-learn` (`LinearRegression`, `r2_score`, `mean_squared_error`), `matplotlib`, and `seaborn`.`
 - Embed a synthetic numeric dataset directly inside the script (e.g., Study Hours vs Marks, or Experience vs Salary).
 - Fit the ``LinearRegression`` model.
 - Plot a scatter plot of raw data points overlaid with the calculated red regression line using ``matplotlib`/`seaborn``.
 - Print the exact calculated slope, intercept, MSE, and R^2 score.
4. STEP-BY-STEP CODE EXPLANATION:
 - Break down data generation, model fitting (``fit()``), coefficient extraction (``coef_``, ``intercept_``), prediction (``predict()``), and plotting step-by-step.
5. EXPECTED OUTPUTS & NUMERICAL RESULTS:
 - Display the expected terminal output values (slope value, intercept value, MSE, R^2 score) and describe the visual appearance of the regression plot.
6. TOP 5 LAB VIVA QUESTIONS & SHORT ANSWERS:
 - Provide 5 viva questions covering regression assumptions, evaluation metrics (R^2 vs MSE), and least-squares optimization with direct answers.

Experiment 5: K-Means Clustering Prompt

Act as a Data Mining lab instructor. Provide a complete academic lab report and functional Python script for Experiment 5: K-Means Clustering and Optimal K Selection via the Elbow Method.

Structure your response with the following exact sections:

1. AIM: A concise 2-sentence objective statement.
2. THEORY & ALGORITHM:
 - Step-by-step K-Means algorithm (Centroid initialization, Euclidean distance assignment, Centroid recalculation, Convergence).
 - Mathematical formula for Euclidean distance and Within-Cluster Sum of Squares (WCSS / Inertia).
 - How the Elbow Method identifies the optimal K value.
3. COMPLETE STANDALONE PYTHON CODE:
 - Use ``pandas``, ``numpy``, ``scikit-learn`` (``KMeans``), ``matplotlib``, and ``seaborn``.
 - Embed a synthetic 2D continuous dataset (e.g., Customer Income vs Spending Score) directly in the code using ``make_blobs`` or a manual dictionary.
 - Loop over K values from 1 to 10, calculate ``inertia_``, and generate an Elbow Plot.
 - Train a final ``KMeans`` model with the chosen optimal K (e.g., K=3).
 - Scatter plot the clusters in 2D space with distinct colors and plot the final cluster centroids with prominent markers (e.g., red 'X' markers).
4. STEP-BY-STEP CODE EXPLANATION:
 - Explain distance computation, inertia calculation, loop iteration over K, final clustering assignment, and plotting calls line-by-line.
5. EXPECTED OUTPUTS & VISUALS:
 - Provide the exact terminal outputs (list of WCSS values for K=1..10, final centroid coordinates, cluster sizes) and describe both the Elbow curve plot and cluster visual.
6. TOP 5 LAB VIVA QUESTIONS & SHORT ANSWERS:
 - Include 5 viva questions on K-Means sensitivity to outliers, initialization methods (K-Means++), and convergence conditions with direct answers.

Experiment 6: Hierarchical Clustering Prompt

Act as a Data Mining lab instructor. Provide a complete academic lab report and functional Python code for Experiment 6: Agglomerative Hierarchical Clustering and Dendrogram Construction.

Structure your response with the following exact sections:

1. AIM: A concise 2-sentence objective statement.
2. THEORY & CONCEPTS:
 - Contrast Agglomerative (bottom-up) vs Divisive (top-down) clustering.
 - Define Linkage criteria with math/logic: Single linkage, Complete linkage, Average linkage, and Ward's method.
3. COMPLETE STANDALONE PYTHON CODE:
 - Use ``pandas`, `numpy`, `scipy.cluster.hierarchy` (`dendrogram`, `linkage`), `scikit-learn` (`AgglomerativeClustering`), and `matplotlib`.`
 - Embed a small synthetic 2D numerical dataset directly inside the script.
 - Generate the linkage matrix using ``scipy.cluster.hierarchy.linkage(X, method='ward')``.
 - Plot a clean Dendrogram visual using ``matplotlib``.
 - Fit ``AgglomerativeClustering(n_clusters=K)`` and output cluster assignments for each point.
4. STEP-BY-STEP CODE EXPLANATION:
 - Explain how pairwise distance matrices are computed, how Ward linkage merges clusters, how dendrogram thresholds work, and how cluster labels are assigned line-by-line.
5. EXPECTED OUTPUTS & DENDROGRAM INTERPRETATION:
 - Provide the numerical linkage output array, cluster assignments array, and a description of reading horizontal cut lines on the resulting dendrogram.
6. TOP 5 LAB VIVA QUESTIONS & SHORT ANSWERS:
 - Provide 5 viva questions on hierarchical tree structures, linkage differences, time complexity, and dendrogram interpretation with short answers.

Experiment 7: Apriori Algorithm Prompt

Act as a Data Mining lab instructor. Provide a complete academic lab report and functional Python code for Experiment 7: Association Rule Mining using the Apriori Algorithm.

Structure your response with the following exact sections:

1. AIM: A concise 2-sentence objective statement.
2. THEORY & MATHEMATICAL FORMULAS:
 - Define Market Basket Analysis concepts: Itemsets, Transactions.
 - State the Apriori Property (Anti-monotonicity principle).
 - Explicit mathematical formulas for core association metrics:
 $\text{Support}(A \rightarrow B) = \text{Count}(A \cup B) / N$
 $\text{Confidence}(A \rightarrow B) = \text{Support}(A \cup B) / \text{Support}(A)$
 $\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) / \text{Support}(B)$
3. COMPLETE STANDALONE PYTHON CODE:
 - Use `pandas` and `mlxtend.frequent_patterns` (``apriori``, ``association_rules``).
 - Create a synthetic transaction dataset directly in the script as a list of lists (e.g., retail store receipts with items like Milk, Bread, Diaper, Beer, Butter).
 - One-hot encode the transactions using `TransactionEncoder` or pandas dummy encoding into a boolean DataFrame.
 - Run ``apriori()`` with ``min_support=0.2`` to generate frequent itemsets.
 - Run ``association_rules()`` with ``metric="confidence"`` and ``min_threshold=0.6``.
 - Print the resulting frequent itemsets table and formatted association rules table.
4. STEP-BY-STEP CODE EXPLANATION:
 - Explain transaction encoding, candidate set generation, support filtering, confidence thresholding, and rule evaluation line-by-line.
5. EXPECTED OUTPUTS & TABLE STRUCTURE:
 - Display the exact output tabular structures (Support for 1-itemsets, 2-itemsets, and final rules table showing antecedents, consequents, support, confidence, and lift metrics).
6. TOP 5 LAB VIVA QUESTIONS & SHORT ANSWERS:
 - Include 5 viva questions on candidate generation pruning, Lift interpretation (Lift > 1 vs Lift < 1), and Apriori vs FP-Growth algorithms with direct answers.